

CSE344 - System Programming

Homework #2 Report

Muhammed Korkmaz
Student ID: 220104004043

March 27, 2026

1 Introduction

This report outlines the design and implementation of an advanced, multi-process POSIX-compatible file search utility. The primary objective of this program is to recursively traverse a specified directory subtree using a parent-worker process model. The utility distributes the search workload among multiple worker processes, filtering files based on a custom regular expression pattern and an optional minimum size constraint. The parent process safely collects the results via signals and inter-process communication, ultimately displaying a well-formatted hierarchical tree and an execution summary.

2 Program Design and Process Management

To effectively manage the workload and strictly prevent buffer overflow issues, a robust Parent-Worker architecture was designed.

- **Directory Partitioning:** The parent process dynamically reads the immediate subdirectories of the target root directory. These subdirectories are then partitioned and assigned to the worker processes in a round-robin fashion. If the root directory contains files directly under it, they are exclusively searched by the parent process to ensure no files are overlooked.
- **Inter-Process Communication (IPC):** Instead of using anonymous pipes, which can suffer from buffer limitations during massive directory traversals, temporary files were utilized. Each worker writes its matched paths and scanned file count to a unique `worker_<PID>.txt` file. The parent reads these files during the summary phase and safely removes them.
- **Memory Management:** To handle exceptionally large directory structures (such as `/usr/share/doc`) without encountering Segmentation Faults, static arrays for subdirectory tracking were replaced with dynamically managed memory using `malloc()` and `realloc()`. All allocated memory is rigorously freed before termination to prevent memory leaks.

3 Signal Handling and Synchronization

Strict adherence to `async-signal-safety` was maintained throughout the implementation. All signal handlers utilize `volatile sig_atomic_t` flags, completely avoiding unsafe functions like `printf()` within the handler context.

1. **SIGUSR1 & SIGCHLD:** Upon task completion, workers send a `SIGUSR1` signal to the parent. The parent captures the sender's PID using the `SA_SIGINFO` flag. A `SIGCHLD` handler using `waitpid()` with `WNOHANG` is implemented to reap terminated children instantly, strictly preventing zombie processes.
2. **SIGINT & SIGTERM:** If the user triggers an interrupt (CTRL-C), the parent catches `SIGINT`, sends `SIGTERM` to all active workers, and waits for a brief period. Workers catch `SIGTERM`, print their partial match counts, and exit cleanly. The parent then aggregates the partial results and terminates.

3.1 Preventing Advanced Signal Race Conditions

A critical synchronization challenge (Lost Wakeup / Signal Merging) occurs when multiple workers finish their tasks in the exact same millisecond. The Linux kernel might merge simultaneous `SIGUSR1` signals, or a `SIGCHLD` might arrive before `SIGUSR1` is fully processed, causing false "unexpected termination" errors.

To definitively solve this, a **Staggered Signal Emission** and **Signal Masking** technique was implemented. Workers calculate a brief delay using `usleep(worker_index * 25000)` before emitting `SIGUSR1`, ensuring signals arrive sequentially. Furthermore, the parent uses `sigprocmask()` to block `SIGUSR1` and `SIGCHLD` before forking, protecting the PID registry array, and uses `sigsuspend()` to safely wait for signals without deadlocking. In addition, workers ignore `SIGINT` directly, leaving the signal catching entirely to the parent process.

4 Pattern Matching Implementation

As the use of external regular expression libraries was prohibited, a custom `match_regex` function was implemented from scratch. The `+` operator correctly acts on the **immediately preceding character** on its left side, as per standard regex conventions.

The algorithm uses a recursive substring matching approach. When a `+` is detected at the next index in the pattern, the function creates two branches of recursion: one that consumes the matching character and stays on the `+` to consume more, and another that moves past the `+`. The comparison is case-insensitive, accomplished by converting both characters to lowercase via `tolower()` before comparison.

5 System Calls and POSIX Functions Used

The program exclusively relies on standard POSIX functions for process management, signal handling, and file operations.

Function	Purpose
<code>fork()</code>	Creates child (worker) processes to distribute the directory traversal workload.
<code>waitpid()</code>	Reaps terminated child processes to prevent zombies and retrieves their exit statuses.
<code>sigaction()</code>	Safely installs signal handlers for <code>SIGINT</code> , <code>SIGTERM</code> , <code>SIGCHLD</code> , and <code>SIGUSR1</code> .
<code>sigprocmask()</code>	Blocks and unblocks signals to protect critical sections and prevent race conditions.
<code>sigsuspend()</code>	Safely puts the parent process to sleep until a specific unmasked signal arrives.
<code>opendir()</code> / <code>readdir()</code>	Opens and reads successive entries from directory streams.
<code>lstat()</code>	Retrieves file metadata (specifically size and type) for filtering criteria.

6 Error Handling

Robust error handling is implemented across all system calls. If a worker encounters an inaccessible directory, it silently skips it to allow uninterrupted processing of valid branches. If a worker terminates unexpectedly (e.g., crashing before sending `SIGUSR1`), the parent's `SIGCHLD` handler detects the missing PID registration and logs a detailed warning to `stderr` without halting the entire program.

7 Test Scenarios and Screenshots

Here are the executions of the mandatory test scenarios proving the stability, correctness, and safety of the implemented utility.

Directory Structure Initialization

The directory structure was meticulously created to match the assignment requirements, containing 5 directories and 9 files.

```
castor@MUHAMMED:~/SYSTEM_HW2$ tree /tmp/crawltest
/tmp/crawltest
├── alpha
│   ├── notes.md
│   ├── report.txt
│   └── subl
│       └── repoort.txt
├── beta
│   ├── error_log.txt
│   ├── image.png
│   └── report_final.txt
├── delta
│   └── summary.txt
├── gamma
│   ├── data.csv
│   └── repooooort.txt
└── 5 directories, 9 files
castor@MUHAMMED:~/SYSTEM_HW2$
```

Figure 1: Initial directory structure verified using the `tree` command.

Test 1: Mandatory Search Scenario

The program accurately builds the tree with correct hierarchical indentation and shows successful regex matching for the given pattern.

```

● castor@MUHAMMED:~/SYSTEM_HW2$ ./procSearch -d /tmp/crawltest -n 3 -f 'rep+ort'
[Worker PID:12133] MATCH: /tmp/crawltest/beta/report_final.txt (0 bytes)
[Worker PID:12133] MATCH: /tmp/crawltest/alpha/report.txt (21 bytes)

/tmp/crawltest
|-- alpha
|----- report.txt (21 bytes) [Worker 12133]
|-- beta
|----- report_final.txt (0 bytes) [Worker 12133]

--- Summary ---
Total workers used : 3
Total files scanned: 14
Total matches found: 2
Worker PID 12133 : 2 matches
Worker PID 12134 : 0 matches
Worker PID 12135 : 0 matches
○ castor@MUHAMMED:~/SYSTEM_HW2$ █

```

Figure 2: Basic search tree and summary output.

Test 2: No Match Scenario

Testing with a non-matching regex (xyz+123). The program does not crash and safely prints the "No matching files found" message with a 0-match summary.

```

● castor@MUHAMMED:~/SYSTEM_HW2$ ./procSearch -d /tmp/crawltest -n 3 -f 'xyz+123'

/tmp/crawltest
No matching files found.

--- Summary ---
Total workers used : 3
Total files scanned: 14
Total matches found: 0
Worker PID 12297 : 0 matches
Worker PID 12298 : 0 matches
Worker PID 12299 : 0 matches
○ castor@MUHAMMED:~/SYSTEM_HW2$ █

```

Figure 3: Execution with no matching files.

Test 3: SIGINT (CTRL+C) and Zombie Process Check

The program successfully catches the SIGINT signal during a massive directory scan, stops all workers safely, prints a partial summary, and leaves zero zombie processes behind.

```

[Worker PID:12477] MATCH: //snap/ubuntu-desktop-installer/1276/usr/lib/x86_64-linux-gnu/libselinux.a (372800 bytes)
[Worker PID:12477] MATCH: //snap/ubuntu-desktop-installer/1276/usr/lib/python3.10/_sysconfigdata__x86_64-linux-gnu.py (27484 bytes)
^C
[Parent] SIGINT received. Terminating all workers...

/
|-- home
|----- castor
|----- .vscode-server
|----- bin
|----- 07ff9d6178ede9a1bd12ad3399074d726ebe6e43
|----- node_modules
|----- @anthropic-ai

```

Figure 4: Graceful shutdown initiated via CTRL+C.

```

----- libselinux1:amd64:symbols (10171 bytes) [Worker 12472]
----- libselinux1:amd64:triggers (72 bytes) [Worker 12472]
----- linux-libc-dev:amd64.list (33120 bytes) [Worker 12472]
----- linux-libc-dev:amd64.md5sums (63692 bytes) [Worker 12472]
----- util-linux.conffiles (93 bytes) [Worker 12472]
----- util-linux.list (10891 bytes) [Worker 12472]
----- util-linux.md5sums (19321 bytes) [Worker 12472]
----- util-linux.postinst (2154 bytes) [Worker 12472]
----- util-linux.postrm (1114 bytes) [Worker 12472]
----- util-linux.preinst (190 bytes) [Worker 12472]
----- util-linux.prerm (790 bytes) [Worker 12472]

--- Partial Summary ---
Total workers used : 8
Total files scanned: 7276
Total matches found: 546
Worker PID 12470 : 0 matches
Worker PID 12471 : 57 matches
Worker PID 12472 : 39 matches
Worker PID 12473 : 0 matches
Worker PID 12474 : 450 matches
Worker PID 12475 : 0 matches
Worker PID 12476 : 0 matches
Worker PID 12477 : 0 matches
castor@MUHAMMED:~/SYSTEM_HW2$ ps -ef | grep procSearch
castor    12528   1871   0 22:52 pts/5    00:00:00 grep --color=auto procSearch
castor@MUHAMMED:~/SYSTEM_HW2$ █

```

Figure 5: Partial summary and process status verifying no zombie processes remain.

8 Conclusion

This implementation successfully fulfills all the requirements of an advanced multi-process file search utility. It accurately distributes workloads, manages complex IPC and signal race conditions, avoids deadlocks, and securely manages dynamic memory. The final output is presented in a clean, hierarchical tree format accompanied by a precise execution summary, demonstrating a deep understanding of Linux system programming concepts.